

Navegação Autónoma na Formula Student: Uma Abordagem Baseada em Aprendizagem por Reforço

Luís Miguel Pereira Silva and Pedro Miguel S. A. Urbano dos Reis

Mestrado em Inteligência Artificial, Universidade do Minho, Braga, Portugal
{pg60390,pg59908}@alunos.uminho.pt

Abstract. Este documento apresenta o planeamento de um trabalho prático centrado no desenvolvimento de um sistema de condução autónoma para a competição Formula Student, recorrendo a técnicas de aprendizagem por reforço. O problema consiste em treinar um agente capaz de navegar numa pista delimitada por cones coloridos, respeitando as regras da categoria *Driverless*. Foi desenvolvido um ambiente de simulação 2D com modelo cinemático de bicicleta, suportando tanto geração procedural como pistas reais de competição em formato YAML. A metodologia assenta na implementação de um agente *Soft Actor-Critic* (SAC) para controlo contínuo, com *Proximal Policy Optimization* (PPO) como *baseline*, e validação assente em *splits train/test* sobre pistas FS reais e ablações controladas.

Keywords: Aprendizagem por Reforço · Condução Autónoma · Formula Student · Soft Actor-Critic · Simulação

1 Introdução e Definição do Problema

A competição Formula Student é a maior competição de engenharia automóvel universitária a nível mundial. A categoria *Driverless* desafia equipas a desenvolver veículos totalmente autónomos capazes de navegar numa pista delimitada por cones coloridos — azuis (esquerda), amarelos (direita) e laranja (partida e chegada) — sem intervenção humana [1]. O veículo deve detetar cones, planejar trajetórias e executar ações de direção e aceleração em tempo real.

A solução proposta assenta em aprendizagem por reforço [2]: treinar um agente que aprenda a conduzir maximizando o progresso e minimizando penalizações. Este relatório documenta a metodologia planeada, incluindo ambiente, algoritmos, ferramentas e protocolo de validação. O código encontra-se em: <https://github.com/pedroreis2468/AR2>.

2 Ambiente de Simulação e Geração de Dados

O treino é conduzido inteiramente em simulação, com um ambiente desenvolvido de raiz seguindo a API *Gymnasium*.

Modelo do veículo. Modelo cinemático de bicicleta [3] referenciado ao centro de gravidade, calibrado para um carro FS típico (~ 250 kg, *wheelbase* de 1.55 m, $\mu = 1.5$). Inclui *rate limiting* no *steering* e restrição de *friction circle* ($a_{\text{lat}}^2 + a_{\text{lon}}^2 \leq (\mu g)^2$). A simulação opera a 50 Hz com *action repeat* de 2 (decisão a 25 Hz).

Pistas: real e procedural. O sistema suporta duas fontes de pistas. (i) **Pistas reais de competição:** 14 traçados oficiais FS em formato YAML obtidos do projeto PacSim (FSG19/21/23/24, FSE22/23/24, FSI24, FSCZ24, FSO20, FSS19, FSS22, entre outros), carregados via um `YAMLTrackLoader` e suavizados com *splines* cúbicas periódicas. (ii) **Geração procedural:** pontos de controlo numa elipse perturbada, suavizados com *splines* fechadas, randomizados a cada episódio — usado como *fallback* e para aumentar diversidade.

Percepção. Sensor de cones (FOV: 180° , alcance: 15 m) que retorna os 3 cones mais próximos de cada cor (azul, amarelo, laranja) no referencial do carro, com ruído gaussiano em distância e *bearing* e probabilidade de deteção de 95%, simulando uma *pipeline* YOLO + *stereo depth*. Cones derrubados são removidos persistentemente do FOV, refletindo o impacto na percepção real do carro.

Regras *Driverless* e colisão. Cones derrubados: +2s por cone azul/amarelo, +4s por cone laranja; deteção de colisão via *bounding box* retangular do veículo. Terminação: ≥ 10 cones derrubados (DOO), desvio > 5 m do *centerline*, ou *off-course* > 2 s consecutivos. Detetam-se também situações de contramão e estagnação prolongada.

3 Metodologia e Algoritmos

O problema é formulado como um MDP com $\gamma = 0.99$:

- **Observação** ($\mathbf{s} \in \mathbb{R}^{24}$): estado do veículo (6 dims — $v_x, v_y, \omega, \delta, a_x, a_y$) e posições relativas dos 3 cones mais próximos por cor (18 dims). O agente *não* recebe a *centerline* nem distâncias às fronteiras — toda a informação de pista provém da percepção local, replicando o cenário real *onboard*.
- **Ação** ($\mathbf{a} \in [-1, 1]^2$): *steering* normalizado e *throttle/brake* unificado.
- **Recompensa:** combinação ponderada de progresso ($\Delta_{\text{prog}} \times 2.0$), *alignment* ($v \cdot \cos(\theta_{\text{err}}) \times 0.4$, com θ_{err} o erro de *heading* face à tangente da *centerline*), suavidade de *steering* com *dead-band* de 0.1 ($\times 0.05$), penalidade lateral linear dentro da pista ($\times 0.08$) e quadrática fora ($\times 2.0$ sobre o excesso de transbordo), custo temporal ($-0.005/\text{step}$) e penalizações por cone (-8 azul/amarelo, -16 laranja). Bónus de volta ($\max(200 - 5n_{\text{cones}}, 50)$) e penalizações terminais (-50) calibram comportamentos limite.

O algoritmo principal é o SAC [4], *off-policy* com maximização de entropia, implementado com *Gaussian actor squashed*, *twin Q-networks* (MLP 256×256), *target networks* com *Polyak* ($\tau=0.005$), *replay buffer* de 1 M e *auto-tuning* da temperatura α (*target entropy* = -0.5 para evitar colapso prematuro). Como *baseline* usa-se o PPO [5] via Stable-Baselines3 [6], com a mesma arquitetura de rede para comparação justa. Aplica-se *domain randomization* [7,8] variando massa ($\pm 15\%$), atrito ($\pm 20\%$) e arrasto ($\pm 10\%$) entre episódios.

4 Ferramentas e Arquitetura de Software

Desenvolvimento em Python (≥ 3.10) com: Gymnasium ≥ 0.29 (API do ambiente), PyTorch ≥ 2.0 (redes *Actor-Critic*), Stable-Baselines3 [6] ≥ 2.1 (PPO e SAC vetorizados), PyGame ≥ 2.5 (renderização 2D), TensorBoard (monitorização), *pytest* (testes unitários) e Git+GitHub (versionamento).

5 Estado Atual da Implementação

A componente principal está implementada e funcional: ambiente Gymnasium completo, com modelo cinemático calibrado, geração procedural, carregador de 14 pistas FS reais, sensor de cones com ruído e *persistent knockdown*, regras FS-AI completas (DOO, *off-course* gradual, bónus de volta); agente SAC *custom* em PyTorch e integração SB3 para SAC e PPO; *scripts* de avaliação com renderização PyGame e *debug* de *reward*. Uma *run* preliminar de SAC (~ 3.8 M *steps*) confirma que o *pipeline* é estável e que o agente aprende, completando voltas em pistas reais. O trabalho restante foca-se no treino comparativo SAC vs. PPO, ablações controladas, análise estatística multi-seed e exploração de *sim-to-sim transfer* para o simulador 3D PacSim. Esta última poderá exigir extensão do espaço de observação para acomodar dinâmica 3D (*pitch/roll*, *slip* de pneus) ou, em alternativa, manter a observação fixa e absorver o *gap* dinâmico via *domain randomization* mais agressiva no ambiente 2D — decisão a determinar empiricamente.

6 Protocolo de Validação

A validação estrutura-se em quatro eixos. (1) **Generalização**: as 14 pistas YAML são divididas em 9 para treino e 5 mantidas inéditas (FSG24, FSI24, FSCZ24, FSE24, FSS22_V2) para teste; reportam-se taxa de voltas completas, cones derrubados, tempo de volta e velocidade média ($\bar{x} \pm \sigma$) por pista. (2) **Eficiência de aprendizagem**: *reward* cumulativa e *episode length* ao longo dos *steps*, agregados sobre 3 *seeds* para o algoritmo principal (SAC) e 1 *seed* para o PPO (*sanity baseline*). (3) **Comparação SAC vs. PPO**: desempenho final, *sample efficiency* e tempo de treino. (4) **Ablações controladas**: contribuição de componentes-chave da metodologia — termo de *alignment*, *domain randomization*, *persistent cone knockdown* e *target entropy* (-0.5 vs. -2.0).

7 Cronograma

Table 1. Planeamento e datas de entrega.

Período	Tarefa
Até 30 abril	Ambiente, agente SAC, <i>pipeline</i> de treino e relatório de planeamento
1–11 maio	Treino SAC (3 <i>seeds</i>) e PPO (1 <i>seed</i>) e <i>reward shaping</i> final
12–18 maio	Ablações controladas e experiências de <i>domain randomization</i>
19–25 maio	Análise estatística de resultados; exploração PacSim
26 maio–1 junho	Preparação da apresentação
2 junho	Apresentação e resultados preliminares
3–15 junho	Incorporação de feedback; redação do relatório final
16 junho	Entrega do relatório final

References

1. Kabzan, J., et al.: AMZ Driverless: The full autonomous racing system. *Journal of Field Robotics* **37**(7), 1267–1294 (2020). <https://doi.org/10.1002/rob.21977>
2. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press, Cambridge, MA (2018). ISBN 978-0-262-03924-6
3. Kong, J., Pfeiffer, M., Schildbach, G., Borrelli, F.: Kinematic and dynamic vehicle models for autonomous driving control design. In: 2015 IEEE Intelligent Vehicles Symposium (IV), pp. 1094–1099. IEEE (2015). <https://doi.org/10.1109/IVS.2015.7225830>
4. Haarnoja, T., et al.: Soft Actor-Critic algorithms and applications. *arXiv:1812.05905* (2018). <https://arxiv.org/abs/1812.05905>
5. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal Policy Optimization algorithms. *arXiv:1707.06347* (2017). <https://arxiv.org/abs/1707.06347>
6. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-Baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research* **22**(268), 1–8 (2021). <https://jmlr.org/papers/v22/20-1364.html>
7. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ IROS, pp. 23–30 (2017). <https://doi.org/10.1109/IROS.2017.8202133>
8. Ulrich, F., Wehrli, T.: End-to-End deep reinforcement learning for autonomous racing dynamics. Bachelor Thesis, ZHAW School of Engineering (2024). <https://doi.org/10.21256/zhaw-32000>